Implementation Guide

LogDLR++: LogLM-Enhanced Cross-System Log Anomaly Detection

Step-by-step implementation task

1. What You Have to Build

Implement an improved version of LogDLR. First reproduce the original LogDLR baseline. Then add the new components one by one. The final model uses LogLM, structure-preserving preprocessing, prototype-aware contrastive alignment, cosine + Huber reconstruction, and conformal calibration.

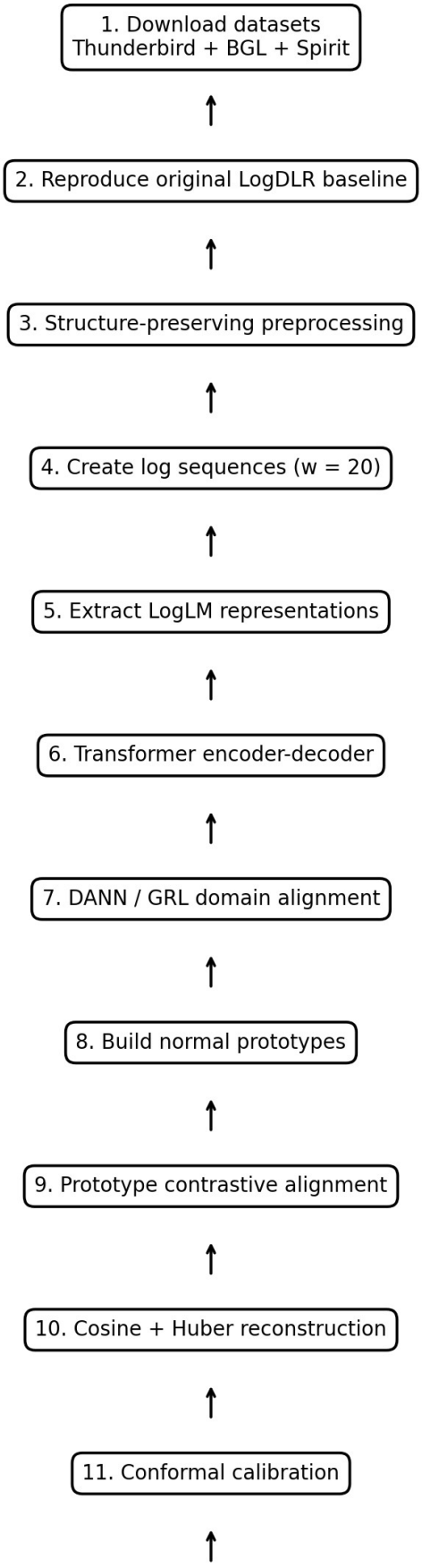

1. Download datasets
Thunderbird + BGL + Spirit
2. Reproduce original LogDLR baseline
3. Structure-preserving preprocessing
4. Create log sequences (w = 20)
5. Extract LogLM representations
6. Transformer encoder-decoder
7. DANN / GRL domain alignment
8. Build normal prototypes
9. Prototype contrastive alignment
10. Cosine + Huber reconstruction
11. Conformal calibration

Figure 1. Overall implementation flowchart.

2. Main Terms in Easy Language

Term	Meaning
Source system	System providing source knowledge.
Target system	Different system on which anomaly detection is tested.
Domain	The system from which the logs come.
Domain-invariant representation	A representation that captures useful behavior while reducing system-specific information.
LogLM	A log-analysis language model. The published LogLM implementation fine-tunes LLaMA-2-7B for multiple log-analysis tasks.
Prototype	Representative vector for a cluster of similar normal log behavior.
DANN / GRL	Adversarial mechanism that reduces source-target domain differences.
Cosine similarity	Measures similarity in vector direction.
Huber loss	Robust reconstruction loss that reduces the effect of very large residuals.
Conformal calibration	Converts anomaly scores into calibrated decisions using a calibration set.
Negative transfer	Source information makes target performance worse.

3. Base Paper and Official Code

The base LogDLR paper uses system-independent preprocessing, SBERT sentence embeddings, a Transformer autoencoder, domain-adversarial training, and reconstruction-error-based anomaly detection.

Official LogDLR implementation:

https://github.com/yingsw79/LogDLR

The repository currently provides dataset references, requirements, single_system.py and cross_system.py.

4. Datasets

Use the same three datasets used in the LogDLR evaluation: Thunderbird, Spirit and BGL. The base paper uses sliding windows of size 20.

Dataset	Role	Source
Thunderbird	Source / target	https://github.com/logpai/loghub
Spirit	Source / target	https://github.com/yingsw79/ LogDLR (Dataset section)
BGL	Source / target	https://github.com/logpai/loghub

LogHub: https://github.com/logpai/loghub

LogHub Zenodo record: https://zenodo.org/records/8196385

Recommended first experiment: Thunderbird -> BGL. After it works, run all other source-target pairs.

5. LogLM Resource

Official LogLM repository:

https://github.com/lunyiliu/LogLM

LogLM paper: https://arxiv.org/abs/2410.09352

Important: do not start by full-fine-tuning LogLM. The first implementation should use a frozen LogLM backbone for representation extraction. If a usable checkpoint is not available, first implement the rest of the pipeline with SBERT as a temporary placeholder, then replace the representation module.

6. Project Folder Structure

```
LogDLR_Project/
├── data/
│   ├── Thunderbird/
│   ├── Spirit/
│   └── BGL/
├── preprocessing/
├── embeddings/
├── models/
├── calibration/
├── experiments/
├── checkpoints/
├── results/
├── configs/
├── scripts/
└── README.md
```

7. Step 1 - Create the Environment

```
conda create -n logdlrpp python=3.10 -y
conda activate logdlrpp
pip install torch torchvision torchaudio
pip install transformers accelerate sentencepiece
pip install numpy pandas scipy scikit-learn
pip install matplotlib seaborn tqdm pyyaml

python -c "import torch; print(torch.__version__); print(torch.cuda.is_available())"
nvidia-smi
```

8. Step 2 - Download and Verify the Data

- Check that all files exist and are readable.
- Check that BGL anomaly/alert information is preserved.
- Check that Thunderbird and Spirit logs are complete.
- Keep each dataset in its own folder.
- Do not train until the data verification succeeds.

9. Step 3 - Reproduce the Original LogDLR Baseline

```
git clone https://github.com/yingsw79/LogDLR.git
cd LogDLR
pip install -r requirements.txt
python single_system.py
python cross_system.py
```

Record Precision, Recall and F1 for Thunderbird -> BGL before changing anything.

10. Step 4 - Structure-Preserving Preprocessing

The proposed preprocessing should retain useful operational information instead of deleting all non-alphabetic content.

Part	Example
Semantic text	connection failed on datanode
Parameters	IP=10.2.1.8, retry=8, port=50010
Structure	timestamp, sequence position, window index

```
Original:
ERROR Node 10.2.1.8 failed after 8 retries on port 50010

Proposed:
semantic = "error node failed after retries"
parameters = [10.2.1.8, 8, 50010]
structure = [timestamp, position, sequence_id]
```

11. Step 5 - Create Log Sequences

Start with the base-paper window size: w = 20.

```
WINDOW_SIZE = 20
for i in range(0, len(logs) - WINDOW_SIZE + 1):
    sequence = logs[i:i+WINDOW_SIZE]
```

Save sequence_id, domain, 20 log messages, and labels for evaluation only.

12. Step 6 - Extract LogLM Representations

For each log message, obtain a hidden representation from LogLM, then pool token states.

```
tokens = tokenizer(log_text, return_tensors="pt",
                   truncation=True)

outputs = loglm(**tokens, output_hidden_states=True)
hidden = outputs.hidden_states[-1]

mask = tokens["attention_mask"].unsqueeze(-1)
embedding = (hidden * mask).sum(dim=1) / mask.sum(dim=1)
```

Use a small projection layer so the LogLM representation matches the sequence encoder dimension.

```
self.projection = nn.Linear(loglm_dim, 384)
```

Cache embeddings to disk:

```
embeddings/
├── Thunderbird_embeddings.pt
├── Spirit_embeddings.pt
└── BGL_embeddings.pt
```

13. Step 7 - Build the Transformer Autoencoder

Use the same basic architecture as LogDLR, but feed LogLM vectors instead of SBERT vectors.

```
Input [batch, 20, embedding_dim]
        ↓
Transformer Encoder
        ↓
Latent representation Z
        ↓
Transformer Decoder
        ↓
Reconstructed sequence X_hat
```

During the LogLM comparison, keep the Transformer architecture unchanged so the comparison is fair.

14. Step 8 - Add DANN / GRL

The domain classifier predicts source or target. The Gradient Reversal Layer makes the encoder learn features that are less dependent on the system identity.

```
Z ———> reconstruction decoder
|
└——> GRL ———> domain classifier
              ├── source
              └── target
```

15. Step 9 - Build Normal Prototypes

A prototype is the center of a cluster of similar normal latent vectors.

```
from sklearn.cluster import KMeans

kmeans = KMeans(n_clusters=4, random_state=42, n_init=10)
cluster_id = kmeans.fit_predict(Z_normal)
prototypes = kmeans.cluster_centers_
```

Start with K=4. Later test K = 2, 3, 4, 5, 6.

16. Step 10 - Add Prototype Contrastive Alignment

```
similarity = cosine_similarity(z, prototypes)
positive = highest_similarity_prototype

L_proto =
    -log(
       exp(sim(z, positive)/tau) /
       sum_k exp(sim(z, prototype_k)/tau)
    )
```

Start with tau = 0.2 and gamma = 0.1.

17. Step 11 - Replace MSE with Cosine + Huber

```
L_cos = 1 - cosine_similarity(X, X_hat)

error = X - X_hat

L_huber =
    0.5 * error**2                  if |error| <= delta
    delta*(|error| - 0.5*delta)     otherwise

L_rec = lambda_cos * L_cos + (1 - lambda_cos) * L_huber
```

Start with lambda_cos = 0.5. Later test 0, 0.25, 0.5, 0.75, 1.0.

18. Step 12 - Final Training Loss

```
L_total =
    L_rec
    + beta * L_domain
    + gamma * L_proto
    + alpha * L_reg
```

Keep every term separate in code. This is required for ablation experiments.

19. Step 13 - Conformal Calibration

Use a held-out calibration set containing normal sequences.

```
p_value(x) =
    (1 + number of calibration scores >= score(x))
    / (n + 1)

if p_value < alpha:
    prediction = ANOMALY
else:
    prediction = NORMAL
```

Start with alpha = 0.05. Later test 0.01, 0.02, 0.05 and 0.10.

Do not use the final test set for calibration.

20. Correct Order of Implementation

1. Reproduce original LogDLR.
2. Replace SBERT with LogLM only.
3. Add structure-preserving preprocessing.
4. Keep/add DANN and GRL.
5. Add normal prototypes.
6. Add prototype contrastive loss.
7. Replace MSE-only reconstruction with cosine + Huber.
8. Add conformal calibration.
9. Run the full model.

21. First Complete Experiment: Thunderbird -> BGL

Experiment	Representation	Alignment	Score	Purpose
E0	SBERT	DANN/GRL	MSE	Original baseline
E1	LogLM	DANN/GRL	MSE	Effect of LogLM
E2	LogLM + structure	DANN/GRL	MSE	Effect of preprocessing
E3	LogLM + structure	DANN + prototype	MSE	Effect of alignment
E4	LogLM + structure	DANN + prototype	Cosine + Huber	Effect of score
E5	Full	DANN + prototype	Cosine + Huber + conformal	Final model

22. Required Cross-System Experiments

Source	Target
Thunderbird	Spirit
Thunderbird	BGL
Spirit	Thunderbird
Spirit	BGL
BGL	Thunderbird
BGL	Spirit

23. Required Ablation Experiments

Ablation	Remove/replace	Purpose
A1	LogLM -> SBERT	Measure LogLM contribution
A2	Remove parameters	Measure preprocessing contribution
A3	Remove prototype loss	Measure prototype contribution
A4	Remove DANN	Measure domain alignment contribution
A5	Cosine only	Measure cosine contribution
A6	Huber only	Measure Huber contribution
A7	MSE only	Direct base-score comparison
A8	Remove conformal	Measure calibration contribution

24. Negative Transfer Experiment

For one target system, compare target-only training against source-to-target training.

```
Delta_F1 = F1(source -> target) - F1(target-only)
```

Repeat with different source systems. Record whether transfer is positive, neutral, or negative.

25. Hyperparameter Sensitivity

Parameter	Initial	Test values
Window size w	20	20, 50, 100, 200
Prototypes K	4	2, 3, 4, 5, 6
Temperature tau	0.2	0.05, 0.1, 0.2, 0.5, 1.0

Cosine weight lambda	0.5	0, 0.25, 0.5, 0.75, 1.0
Prototype weight gamma	0.1	0, 0.01, 0.05, 0.1, 0.5, 1.0
Domain weight beta	1.0	0, 0.1, 0.5, 1.0, 2.0
Conformal alpha	0.05	0.01, 0.02, 0.05, 0.10

26. Evaluation Metrics

Metric	Purpose
Precision	Correctness of detected anomalies
Recall	How many real anomalies are detected
F1-score	Balance between precision and recall
PR-AUC	Useful for imbalanced anomaly detection
AUROC	Ranking quality over thresholds
False Positive Rate	Unnecessary alert rate
Inference time	Practical efficiency
GPU memory	Computational requirement

27. Required Results

Result	What to include
Overall performance	All source-target pairs
Ablation	Every proposed component
Reconstruction comparison	MSE, cosine, Huber, hybrid
Calibration comparison	F1 threshold, percentile, conformal
Negative transfer	Source-to-target vs target-only
Sensitivity	K, tau, lambda, gamma, beta, alpha
Efficiency	Runtime and GPU memory

28. Required Figures

- Overall architecture.
- Normal vs anomaly score distributions.
- Latent-space visualization before and after alignment.
- Prototype cluster visualization.
- Ablation F1-score comparison.
- Calibration/shift robustness plot.
- Hyperparameter sensitivity plots.

29. Minimum Code Modules

File	Task
data_loader.py	Load raw logs and labels
preprocess.py	Structure-preserving preprocessing
sequence_builder.py	Create windows
loglm_encoder.py	LogLM hidden-state extraction
baseline_model.py	Original LogDLR baseline
domain_adversarial.py	GRL + domain classifier
prototype.py	Prototype generation
contrastive_loss.py	Prototype contrastive loss

reconstruction_loss.py	MSE/cosine/Huber/hybrid
conformal.py	Calibration and decision
train.py	Training loop
evaluate.py	Metrics
run_experiments.py	Ablation/cross-system experiments

30. What Not to Do

- Do not tune on the final test set.
- Do not change the data split after seeing the result.
- Do not report only the best source-target pair.
- Do not claim LogLM is better without a controlled SBERT comparison.
- Do not remove the original LogDLR baseline.
- Do not full-fine-tune the 7B LogLM model as the first step.
- Do not add all new losses at the same time.
- Do not insert results that were not actually obtained.

31. Final Deliverables

10. Complete source code and README.
11. requirements/environment file.
12. Preprocessing scripts.
13. Cached LogLM embeddings.
14. Reproduced LogDLR baseline.
15. Proposed model checkpoint.
16. Configuration files.
17. Overall performance table.
18. Ablation table.
19. Cross-system results.
20. Negative-transfer analysis.
21. Sensitivity plots.
22. Latent-space and anomaly-score visualizations.
23. Short implementation report.

32. Milestones

Phase	Task	Done when
1	Environment + data	All datasets readable
2	Baseline	Original LogDLR runs
3	LogLM	Embeddings are cached
4	Representation	LogLM + preprocessing works
5	Alignment	DANN + prototype loss works
6	Scoring	Cosine + Huber works
7	Calibration	Conformal decision works
8	Evaluation	All experiments completed

33. Source References

LogDLR implementation: https://github.com/yingsw79/LogDLR

LogLM implementation: https://github.com/lunyiliu/LogLM

LogLM paper: https://arxiv.org/abs/2410.09352

LogHub: https://github.com/logpai/loghub

LogHub Zenodo: https://zenodo.org/records/8196385

This implementation guide is based on the uploaded LogDLR base paper and the proposed LogDLR++ research design. The base paper uses a Transformer autoencoder, SBERT, DANN/GRL, and reconstruction-error-based anomaly detection; the proposed implementation adds the new modules described in this guide.